1.0 Introduction

1.1 Executive Summary

This project proposes the design and development of a relational database system for the International Student Support Unit (ISSU) at Albukhary International University (AIU). ISSU plays a critical role in managing international students throughout their study lifecycle, including admission, visa and insurance matters, and final exit from the university.

1.2 Organization Background

Albukhary International University (AIU) is a residential university that hosts a large number of international students. The International Student Support Unit (ISSU) is responsible for:

- Managing student records and personal information
- Handling visa applications for prospective students.
- Handling visa renewals and immigration-related processes
- Handling student exits (completion, deferral, or progression)

1.3 Current System and Problems

Currently, many ISSU processes are managed using a combination of:

- Paper forms (printed applications, clearance forms, etc.)
- Spreadsheets (Excel files for student lists, visa expiry tracking master sheet, insurance lists)
- Email communication and manual updates

This leads to several issues:

- Data fragmentation: information is spread across multiple files and formats
- Redundant data entry: the same student details are retyped for different services
- Human errors: incorrect or outdated student information and visa dates
- No centralized tracking: it is difficult to see a student's full status (visa, insurance, exit) in one place
- Slow processing: manual checking of expiry dates and eligibility causes delays
- Compliance risks: mistakes in visa or insurance tracking can affect immigration compliance

1.4 Benefits of the System

The ISSU Management System is a web-based platform that enables students to manage their visa, insurance, and exit processes while receiving automated reminders

before visa expiry. ISSU staff can efficiently manage student profiles, process visa renewals, handle insurance and exit cases, and generate key reports through a centralized dashboard. The system uses a secure layered architecture to ensure high performance, data protection, and ease of use for both students and staff.

2.0 Project Objectives

The primary objective of this project is to design and develop a centralized, reliable, and secure relational database system that supports the operational responsibilities of the International Student Support Unit (ISSU) at Albukhary International University. The specific objectives of the project are as follows:

2.1.To centralize international student data into a single relational database system

This objective aims to eliminate data fragmentation caused by paper forms, spreadsheets, and scattered email records by storing all student, visa, insurance, and exit information in one integrated database. Success is measured by the ability to retrieve a complete student record (profile, visa, insurance, and exit status) from a single system interface.

2.2. To reduce redundant data entry and minimize human errors in ISSU processes

The system seeks to ensure that student information is entered once and reused across visa, insurance, and exit workflows. This objective is achieved by enforcing normalized database design and shared data structures, thereby reducing inconsistencies such as duplicated or outdated records.

2.3. To support efficient visa and insurance management throughout the student lifecycle

The project aims to provide structured workflows for visa applications, renewals, and insurance management, allowing both students and staff to submit, process, and track requests digitally. Effectiveness is measured by the system's ability to track application status, store supporting documents, and update official records accurately.

2.4. To automate expiry tracking and reminder generation to improve compliance

This objective focuses on reducing compliance risks by automatically identifying visas and insurance policies approaching expiry and generating reminders in advance. The system's success is measured by accurate classification of expiry periods and prevention of missed or duplicate reminders.

2.5. To enable comprehensive tracking of student exit and clearance processes

The project aims to digitize exit requests and clearance workflows, ensuring that all required steps (exit cases, unit clearances, and exit visa actions) are recorded and traceable. Completion of this objective is measured by the system's ability to display exit progress and prevent incomplete or inconsistent exit records.

2.6. To provide ISSU staff with real-time operational visibility through dashboards and reports

The system is designed to support informed decision-making by presenting staff with dashboards and reports summarizing student status, visa and insurance expiry risks, pending renewals, and exit cases. This objective is met when staff can access up-to-date summaries and detailed reports without relying on manual spreadsheets.

2.7. To enhance processing efficiency and reduce administrative delays

By replacing manual checks and paper-based workflows with automated queries, filters, and status updates, the system aims to significantly reduce processing time for common ISSU tasks. Success is measured by faster identification of pending cases and reduced dependency on manual tracking.

2.8. To ensure data security, integrity, and controlled access to sensitive information

The project aims to protect student and staff data through role-based access control, secure authentication, and enforcement of database constraints. This objective is achieved when only authorized users can access or modify data, and when the system prevents unauthorized or inconsistent updates.

2.9. To improve usability and accessibility for both students and ISSU staff

The system seeks to provide intuitive web interfaces that allow non-technical users to complete tasks such as profile updates, application submissions, and case processing with minimal training. Effectiveness is measured by the ability of users to complete key workflows without external tools or manual intervention.

2.10.. To establish a scalable and maintainable foundation for future ISSU system enhancements

The project aims to deliver a modular and well-structured database and system design that can be extended to support additional ISSU functions or increased student numbers. This objective is met when new features can be added without major redesign of existing data structures.

3.0 System Requirements Specification (SRS)

3.1. Functional Requirements

FR-1: User authentication and role enforcement

1.1 The system shall allow users to authenticate using a valid email and password.
1.2 The system shall differentiate between student and staff users at login.
1.3 The system shall redirect authenticated users to the appropriate portal based on role.
1.4 The system shall prevent students from accessing staff-only pages.
1.5 The system shall prevent staff from accessing student-only pages.
1.6 The system shall allow authenticated users to log out and terminate their session.

FR-2: Session management

2.1 The system shall create a session upon successful authentication.
2.2 The system shall store user role information within the session.
2.3 The system shall enforce an inactivity timeout on all authenticated sessions.
2.4 The system shall invalidate the session upon logout.
2.5 The system shall prevent reuse of an invalidated or expired session.

FR-3: Student registration and profile creation

3.1 The system shall allow students to register by submitting required personal and academic details.
3.2 The system shall validate registration inputs for completeness, format, and uniqueness.
3.3 The system shall ensure that each student is assigned to **exactly one** academic subtype (PC, UG, or PG).
3.4 The system shall store the student's program and school information.
3.5 The system shall create the student profile and related subtype records as a single consistent operation.

FR-4: Student profile management

4.1 The system shall allow students to view their complete profile information.
4.2 The system shall allow students to update editable profile fields.
4.3 The system shall allow students to upload or update a profile photograph.
4.4 The system shall validate profile updates before saving changes.
4.5 The system shall display updated profile information immediately after successful updates.

FR-5: Nationality management

5.1 The system shall allow a student to be associated with one or more nationalities.
5.2 The system shall allow one nationality to be designated as the primary nationality, where applicable.
5.3 The system shall allow nationality information to be updated through the student profile.
5.4 The system shall store nationality data using a normalized relationship structure.

FR-6: Staff student record management (search, filter, sort, view)

6.1 The system shall allow staff to view a list of all registered students.
6.2 The system shall allow staff to search student records by identifiers such as student ID, name, email, or passport number.
6.3 The system shall allow staff to filter student records by attributes such as student type, academic status, visa status, and insurance status.
6.4 The system shall allow staff to sort student records by fields including name, student ID, visa expiry date, and insurance expiry date.
6.5 The system shall allow staff to view detailed student records, including visa, insurance, and exit case information.
6.6 The system shall allow staff to update permitted administrative fields within a student record.

FR-7: Visa record management

7.1 The system shall store visa records for students, including issue date, expiry date, passport number, and visa status.
7.2 The system shall allow students to view their current visa information.
7.3 The system shall allow staff to create a visa record if none exists for a student.
7.4 The system shall allow staff to update visa records following renewal or correction.
7.5 The system shall allow staff to view historical visa records where applicable.

FR-8: Visa renewal application workflow

8.1 The system shall allow students to submit visa renewal applications.
8.2 The system shall require each visa renewal application to be linked to a specific student.
8.3 The system shall allow staff to create or initiate a visa renewal application on behalf of a student.
8.4 The system shall allow staff to review visa renewal applications.
8.5 The system shall allow staff to update the status of visa renewal applications.
8.6 The system shall allow staff to record remarks for each status update.
8.7 The system shall maintain a chronological renewal status history.

FR-9: Visa document management

9.1 The system shall allow students to upload visa renewal documents.
9.2 The system shall allow uploaded documents to be updated or replaced.
9.3 The system shall allow authorized deletion of visa documents.
9.4 The system shall validate uploaded documents for file type and size.
9.5 The system shall ensure all documents are correctly linked to the relevant application and student.

FR-10: Reminder generation and opt-out handling

10.1 The system shall automatically generate visa expiry reminders based on predefined time thresholds.
10.2 The system shall allow students to opt out of visa expiry reminders.
10.3 The system shall exclude ineligible students from reminder generation.
10.4 The system shall prevent duplicate reminders for the same visa and due date.
10.5 The system shall store generated reminders in a reminder queue for tracking and reporting.

FR-11: Insurance management

11.1 The system shall allow students to view their insurance policy information.
11.2 The system shall allow students to submit insurance claims.
11.3 The system shall allow students to submit insurance renewal requests.
11.4 The system shall allow staff to initiate insurance renewal records on behalf of students.
11.5 The system shall allow staff to process insurance claims and renewals by updating status and remarks.
11.6 The system shall allow staff to view lists of insurance policies nearing expiry.

FR-12: Exit request and clearance workflow

12.1 The system shall allow students to submit exit requests.
12.2 The system shall allow staff to initiate exit cases on behalf of students.
12.3 The system shall allow staff to update exit case status.
12.4 The system shall allow staff to create and manage clearance records.
12.5 The system shall allow staff to record unit clearances with approval details.
12.6 The system shall allow staff to record exit visa actions.
12.7 The system shall allow students to view exit case status and progress.

FR-13: Notifications

13.1 The system shall generate notifications related to visa, insurance, and exit processes.
13.2 The system shall allow students to view received notifications.
13.3 The system shall track unread and read notification states.
13.4 The system shall allow staff to send manual notifications to students.

FR-14: Reporting and dashboards

14.1 The system shall provide staff dashboards summarizing operational information.
14.2 The system shall allow staff to generate visa expiry reports.
14.3 The system shall allow staff to generate insurance expiry reports.
14.4 The system shall allow staff to view visa renewal and exit case status reports.
14.5 The system shall allow reports to be filtered and sorted by relevant attributes.

FR-15: Profile management

15.1 The system shall allow users to view and update profile details, including first name, last name, phone number, and department (for staff) or equivalent profile attributes (for students).
15.2 The system shall validate all profile updates before saving changes.
15.3 The system shall save profile changes and reflect them immediately upon successful update.

FR-16: Profile photo management

16.1 The system shall allow users to upload a profile photo.
16.2 The system shall allow users to edit the uploaded photo, including basic operations such as zooming, rotating, and cropping before saving.
16.3 The system shall validate uploaded images for file type and size.
16.4 The system shall associate the saved profile photo with the user account and display it across the system where applicable.

FR-17: Password management

17.1 The system shall allow users to change their account password from the settings page.
17.2 The system shall require users to enter their current password before setting a new password, except where the current password is not yet set (e.g., first-time setup).
17.3 The system shall enforce password rules, including minimum length and character requirements.
17.4 The system shall require confirmation of the new password before applying the

change.
17.5 The system shall securely update the stored password upon successful validation.

FR18: Language and localization support

18.1 The system shall allow users to select their preferred interface language from the account or navigation menu.
18.2 The system shall support the following interface languages:

- English
- Bahasa Melayu
- Bahasa Indonesia
- Burmese
- Arabic
- Sinhala

18.3 The system shall update the user interface language immediately after selection.
18.4 The system shall persist the selected language preference for future sessions.

FR-19: Logout functionality

19.1 The system shall allow users to log out from the account settings or navigation menu.
19.2 The system shall invalidate the user session upon logout.
19.2 The system shall redirect the user to the login page after successful logout.

3.2 Non-Functional Requirements

NFR-1: Security

The system shall store all user passwords using secure cryptographic hashing and shall never store or transmit passwords in plaintext. Role-based access controls shall be enforced on all protected resources.
Applies to: Authentication module, all restricted pages.

NFR-2: Data integrity and consistency

The system shall enforce data integrity using primary keys, foreign keys, uniqueness constraints, and transactional operations. Multi-step workflows shall either complete fully or be rolled back.
Applies to: Database schema, stored procedures.

NFR-3: Reliability

The system shall handle database and application errors gracefully without exposing internal system details to users and shall maintain consistent system state during failures.
Applies to: Application logic, database operations.

NFR-4: Performance

The system shall provide acceptable response times for typical student and staff operations and shall optimize recurring reports and date-based queries using indexed fields and stored procedures.
Applies to: Reporting modules, visa and insurance queries.

NFR-5: Usability

The system shall provide clear feedback messages for user actions and shall present interfaces suitable for non-technical users in an administrative university context.
Applies to: Student and staff portals.

NFR-6: Maintainability

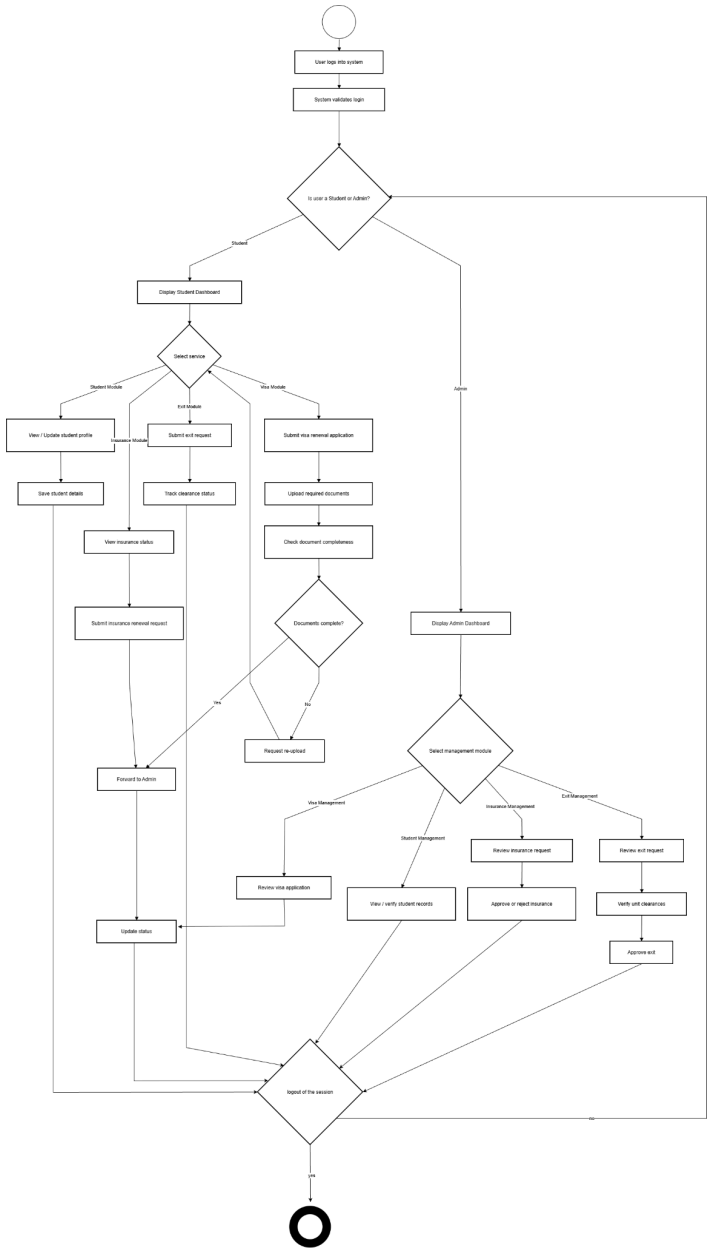
The system shall be modularly structured such that changes to one functional area (e.g., visa management) do not require extensive modification to unrelated components.
Applies to: Overall system architecture.

NFR-7: Compatibility and deployment

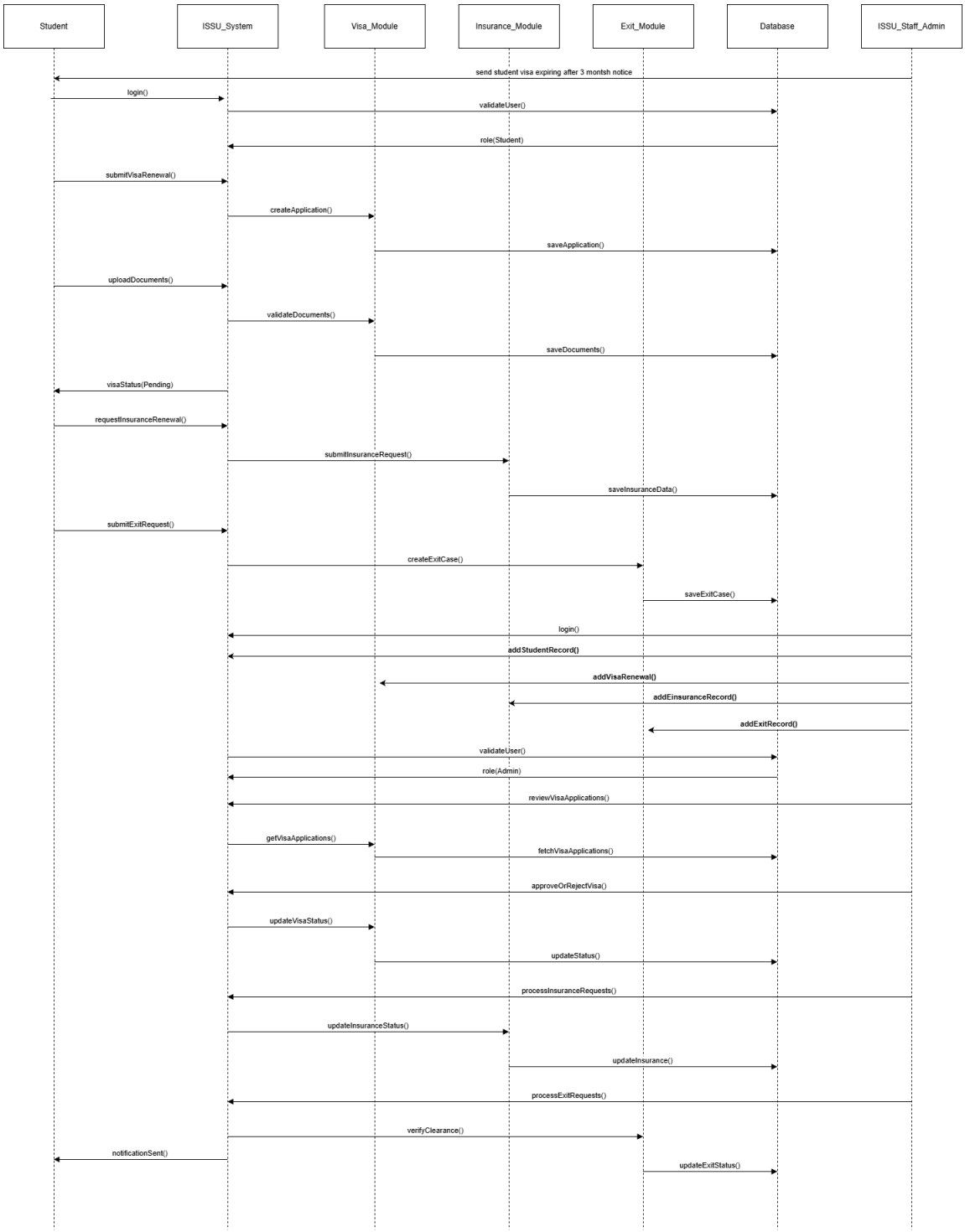
The system shall be deployable on a standard PHP and MariaDB environment and shall be compatible with modern web browsers.
Applies to: Deployment environment.

4.0 System Model

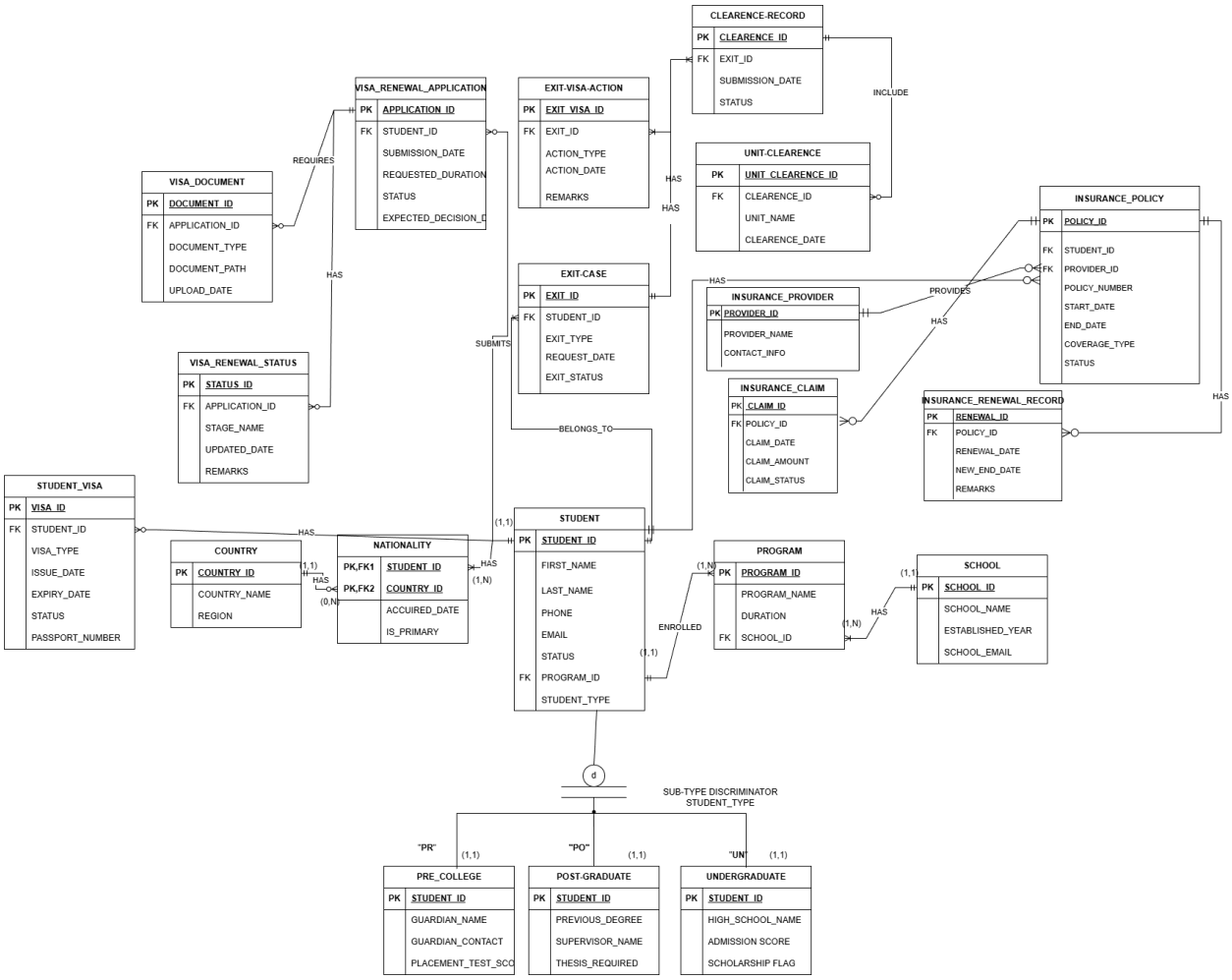
4.1. Activity Diagram



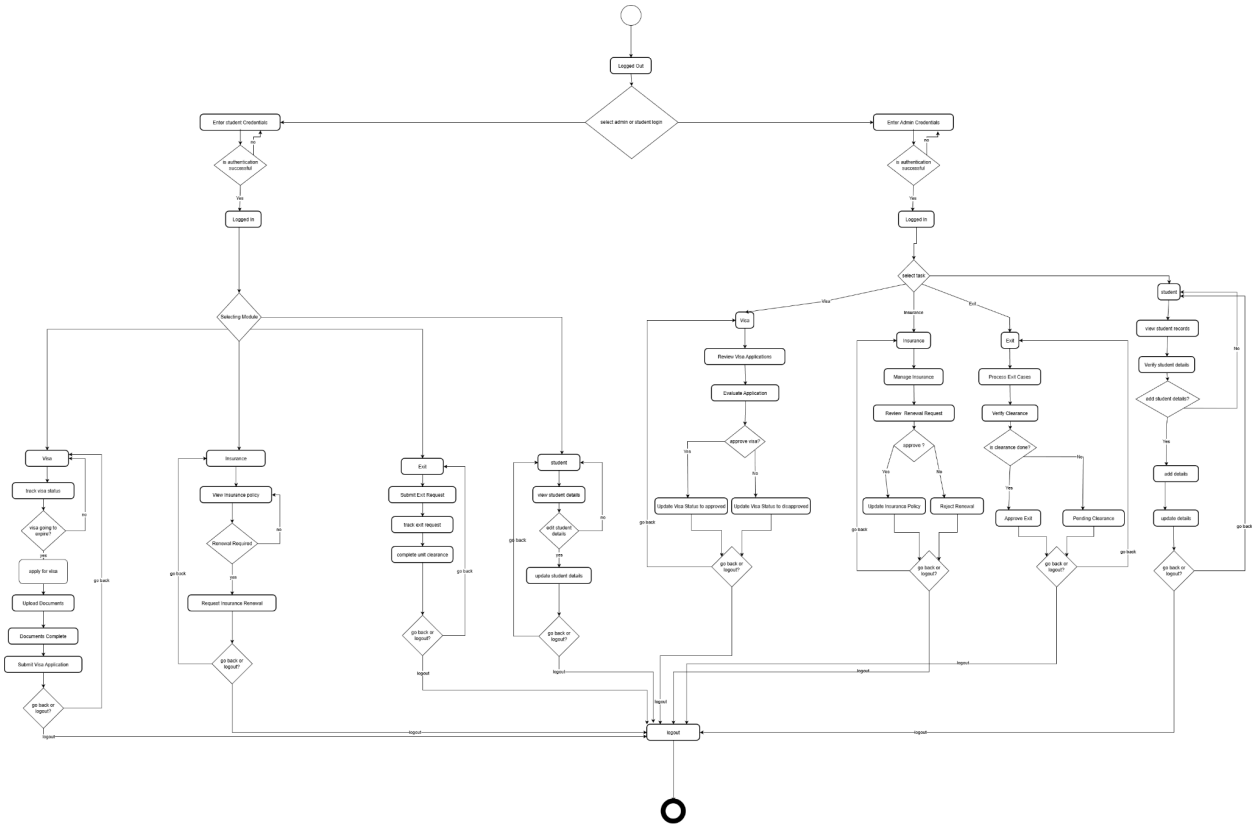
4.2. Sequence Diagram



4.3. Class Diagram



4.4. State Diagarm



5.0 System Architecture design

The system architecture follows a three-tier monolithic web architecture consisting of a presentation layer, an application layer, and a data layer. The presentation layer is implemented as separate student and staff web portals, providing role-specific user interfaces and enforcing access boundaries. The application layer, implemented in PHP, is responsible for session management, role-based authorization, input validation, workflow coordination, and interaction with the database. The data layer is implemented using a relational MariaDB database, which stores all persistent data and enforces structural integrity through primary keys, foreign keys, uniqueness constraints, and controlled relationships. This architectural style was selected because it provides simplicity of deployment, clear separation of concerns, and suitability for a medium-scale institutional system managed by a small development team.

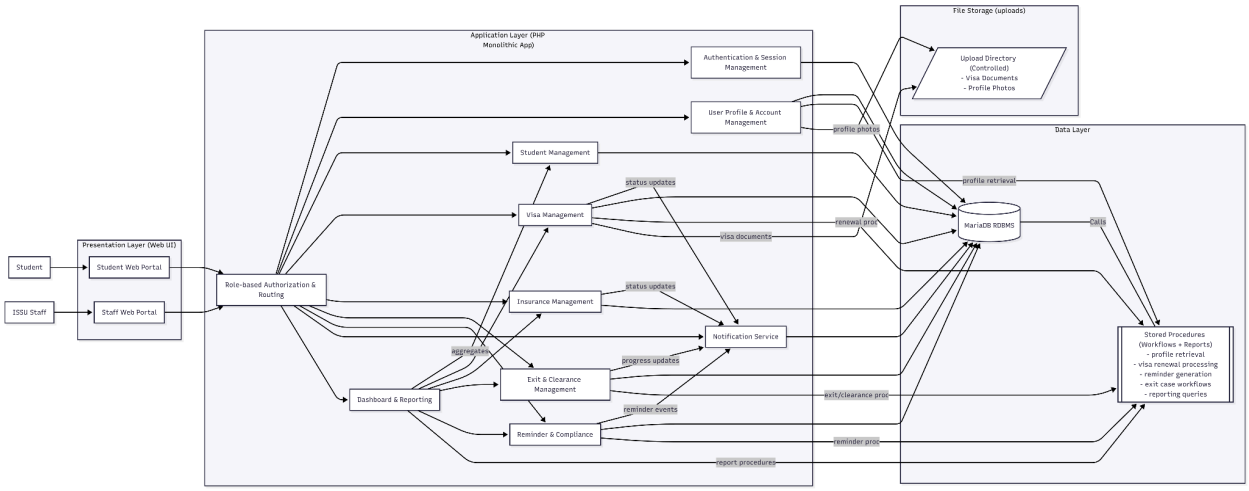
A key architectural decision is the use of stored procedures within the database for critical workflows and reporting operations. Stored procedures are employed for multi-step processes such as student profile retrieval, visa renewal processing, reminder generation, exit case management, and institutional reports. This approach centralizes business rules, ensures transactional consistency across multiple tables,

and reduces duplication of logic across application code. It also enables reliable rollback in the event of partial failures and improves performance for complex queries that are frequently executed. While this increases reliance on the database layer, the trade-off is justified by the need for strong data integrity, consistency, and predictable behavior in administrative processes.

Security is treated as a core architectural concern. Authentication is session-based, with all passwords stored using secure hashing mechanisms and never in plaintext. Authorization is enforced by role-specific routing and server-side access checks, ensuring that students and staff can only access functions appropriate to their roles. All user input is validated server-side to prevent malformed data and unauthorized actions. File uploads, such as visa documents and profile photos, are restricted by file type and size and stored in controlled directories to reduce security risk. Session expiration mechanisms are used to mitigate unauthorized access due to inactivity.

From a modular perspective, the system is logically divided into clearly defined functional components, including authentication, student management, visa management, insurance management, exit management, notifications, and reporting. Each module encapsulates a distinct set of responsibilities and interacts with others through well-defined data flows. Although the system is deployed as a single application, this logical modularity improves maintainability, supports future enhancement, and allows individual components to be reasoned about independently during analysis and testing.

Overall, the architectural design balances simplicity, robustness, and institutional suitability. The three-tier structure provides clarity and separation of responsibilities, the database-centric workflow enforcement ensures correctness and auditability, and the modular decomposition aligns closely with real-world ISSU processes. The system architecture is therefore well-organized, justified by the operational context, and sufficiently flexible to support future extensions without fundamental redesign.



6.0 Component-Based Model

The ISSU Management System follows a component-based model where the application is decomposed into cohesive, loosely coupled components. Each component encapsulates a focused set of responsibilities and communicates with other components only through explicit interfaces.

To support modularity and low coupling, every component is defined using:

- Provided interfaces (P): Services the component offers to other components.
- Required interfaces (R): Services the component depends on from other components.

Although the system is deployed as a single PHP web application, the logical component separation ensures maintainability, reuse, and controlled dependencies. Components coordinate through interface calls and share access to a centralized relational database and stored procedures.

6.1 Authentication and Session Management Component

Responsibilities: login, password validation, session creation/validation, logout, role-based access enforcement.

Provided Interfaces (P):

- IAuthService (login/logout)
- ISessionService (create/validate session)

- IAccessControl (role/permission checks)

Required Interfaces (R):

- IUserAccountData (retrieve account credentials/roles from profile/account storage)

Reuse potential: usable across other university portals requiring standard authentication + RBAC.

6.2 User Profile and Account Management Component

Responsibilities: manage profile details, profile photos, password updates, language preferences, account settings.

Provided Interfaces (P):

- IUserAccountData (account identity + credentials metadata access)
- IProfileService (read/update profile)
- IPasswordService (change password)
- IPhotoUploadService (upload/retrieve profile photo)
- ILanguagePreference (persist localization settings)

Required Interfaces (R):

- IAuthContext (validated user identity/session context from authentication)

Reuse potential: generic profile, password, localization patterns applicable to most multi-user systems.

6.3 Student Management Component

Responsibilities: student registration, subtype assignment (PC/UG/PG), nationality, status tracking, core student record storage.

Provided Interfaces (P):

- IStudentService (create/read/update student profile)
- IStudentLookup (search/filter basic student identity)
- IStudentStatus (retrieve status/subtype/nationality)

Required Interfaces (R):

- IAuthContext (staff/student authorization context)

Reuse potential: student entity handling reusable across other academic administration systems.

6.4 Visa Management Component

Responsibilities: visa records, renewal applications, structured status workflow, visa document handling.

Provided Interfaces (P):

- IVisaService (visa CRUD)
- IVisaRenewalWorkflow (apply/review/approve/reject transitions)
- IVisaDocumentService (upload/validate/retrieve visa docs)
- IVisaExpiryData (expiry dates for reminders/reporting)

Required Interfaces (R):

- IStudentLookup / IStudentStatus (student identity + eligibility fields)
- INotificationPublisher (emit status updates)
- IFileStorage (store/retrieve uploaded documents)

Reuse potential: approval workflow and document pipeline reusable for other administrative renewals.

6.5 Reminder and Compliance Component

Responsibilities: detect upcoming visa/insurance expiries, generate reminders, enforce opt-out rules, prevent duplicates.

Provided Interfaces (P):

- IReminderService (create/manage reminders)
- IComplianceRules (opt-out + duplicate prevention policies)

Required Interfaces (R):

- IVisaExpiryData (visa expiry inputs)

- IInsuranceExpiryData (insurance expiry inputs)
- INotificationPublisher (deliver reminders)
- IReportingFeed (expose reminder metrics)

Reuse potential: generic time-based compliance engine for deadlines, fee reminders, etc.

6.6 Insurance Management Component

Responsibilities: policies, renewals, claims, structured status tracking.

Provided Interfaces (P):

- IInsuranceService (policy CRUD)
- IInsuranceWorkflow (renewal/claim status transitions)
- IInsuranceExpiryData (expiry dates for reminders/reporting)

Required Interfaces (R):

- IStudentLookup (ownership/identity)
- INotificationPublisher (updates to users)
- IReportingFeed (dashboard/report contribution)

Reuse potential: claims/renewal workflow reusable for other entitlement management modules.

6.7 Exit and Clearance Management Component

Responsibilities: exit requests, exit cases, clearance records, unit clearances, exit visa actions, traceable multi-step process.

Provided Interfaces (P):

- IExitService (exit requests/cases)
- IClearanceWorkflow (multi-unit clearance progression)
- IExitStatusTracking (audit/status history)

Required Interfaces (R):

- IStudentLookup (identity)
- INotificationPublisher (progress updates)
- IReportingFeed (operational metrics)

Reuse potential: multi-step clearance model reusable for graduation or departmental approvals.

6.8 Notification Component

Responsibilities: generate/store/display notifications; unread/read tracking; staff-initiated messages; dashboard display.

Provided Interfaces (P):

- INotificationPublisher (publish event-based notifications)
- INotificationInbox (query and render user notifications)
- INotificationStatus (read/unread tracking)

Required Interfaces (R):

- IAuthContext (target user identity)
- IUserDirectory (resolve user recipients by role or id)

Reuse potential: generic alert system for any administrative information system.

6.9 Dashboard and Reporting Component

Responsibilities: aggregate operational data; dashboards; drill-down reports; search/filter/sort.

Provided Interfaces (P):

- IReportingService (generate reports)
- IDashboardService (KPIs, summaries, drill-down links)

Required Interfaces (R):

- IStudentService / IStudentLookup
- IVisaService / IVisaExpiryData
- IInsuranceService / IInsuranceExpiryData
- IExitService
- IReminderService
- INotificationInbox
- IReportProcedures (stored procedures for heavy queries)

Reuse potential: reporting patterns reusable for other departmental dashboards.

7.0 Testing Strategy

7.1. Testing objectives

Testing will ensure that:

7.1.1. Every functional requirement behaves as specified for both roles.

7.1.2. Business rules are enforced at both the application and database layers.

7.1.3. Stored procedures and triggers produce correct results and do not break PHP execution.

7.1.4. Tthe system protects sensitive data and prevents unauthorized access.

7.1.5. File uploads are safe and correctly associated with the correct student/application;

7.1.6. Reporting queries return correct outputs and remain performant; and

7.1.7 The overall system remains usable and stable under normal and edge-case conditions.

7.2. Testing levels, methods, and expected outcomes

7.2.1 Unit testing

Unit testing focuses on verifying isolated behaviors in the PHP application logic and any reusable helper functions (validation, session checks, file upload handlers).

The main method is white-box testing using direct execution of functions and controlled inputs, supplemented by manual code-path tests where automated unit frameworks are not available.

Expected outcomes include correct validation decisions, correct session enforcement behavior, correct input sanitization behavior, and correct file-handling decisions (accepted vs rejected based on policy). Critical unit tests include email format checks, password rules, role enforcement checks, file extension validation, file size validation, and safe filename generation.

7.2.2 Database procedure testing

Since the system relies on stored procedures for workflows (reminders, reports, exit-case handling, status updates, profile retrieval), each procedure must be tested independently at the database layer.

The method is black-box database testing: execute each stored procedure with carefully designed inputs, verify expected row outputs, verify table changes, and verify transaction behavior (commit/rollback).

Expected outcomes include correct inserts/updates/deletes, consistent foreign key behavior, correct exclusion rules (opt-out/graduated), duplicate-prevention logic working as intended, and correct handling of edge cases (no eligible records, multiple visas, missing optional records). Transaction tests must explicitly confirm that partial operations do not persist when an error occurs.

7.2.3 Integration testing

Integration testing verifies that PHP pages correctly call database queries/procedures, correctly process result sets, correctly handle stored procedure multi-results, and correctly store/retrieve file uploads.

The method is scenario-based testing: execute a complete request in the browser and validate the resulting database state and displayed UI outcome.

Expected outcomes include correct data persistence, correct page behavior, correct linking between tables (e.g., renewal documents linked to correct application), correct error handling when database constraints are violated, and correct cleanup behavior in delete operations. Integration tests must also confirm that file uploads are not “orphaned” (file stored but DB row missing) and that DB rows do not refer to missing files.

7.2.4 System testing

System testing validates full real workflows as users would perform them.

The method is end-to-end test execution using predefined test personas and data.

Expected outcomes include successful completion of processes with correct transitions, correct permissions at each step, correct notifications and reporting updates, and correct enforcement of business rules. This stage includes the most important tests because the system’s value comes from workflow correctness.

Key end-to-end workflows to test include:

- Student registration → subtype assignment → nationality assignment → profile view correctness
- Student submits visa renewal → uploads documents → staff reviews → status updates → student sees timeline

- Reminder generation procedure → reminder queue correctness → opt-out exclusion → duplicate prevention
- Student submits insurance claim → staff processes → student sees updated status
- Student submits exit request → staff creates clearance → unit clearances recorded → actions logged → case closes

7.2.5 User acceptance testing (UAT)

UAT verifies that the system matches ISSU operational expectations and that workflows align with the real administrative process. The method is guided testing sessions with staff and student representatives using realistic tasks and checklists. Expected outcomes are that users can complete tasks without confusion, the outputs match institutional expectations, reports are meaningful, and staff agree that the approval steps reflect their real work. UAT also captures usability issues such as unclear messages, missing field guidance, confusing document requirements, and workflow gaps.

7.3. Test coverage

7.3.1. Authentication & session control

Methods: role-based access tests, session timeout tests, invalid credential tests.
Expected outcomes: only correct users can access their portal; session expires as configured; logout fully invalidates session; unauthorized page access is blocked.

7.3.2. Student profile, subtype, and nationality

Methods: CRUD tests, integrity tests.
Expected outcomes: student always has exactly one subtype; multiple nationalities are correctly stored and displayed; profile changes persist correctly; invalid inputs are rejected.

7.3.3. Visa and visa renewal workflow

Methods: workflow tests, edge-case tests (no visa record, expired visa, multiple visas).
Expected outcomes: renewal submissions create correct application records; documents attach correctly; staff updates are reflected to students; invalid transitions are blocked; expiry logic remains correct.

7.3.4. File uploads (visa docs and photos)

Methods: allowlist/denylist tests, size limit tests, path traversal tests.
Expected outcomes: only allowed types upload; disallowed types are rejected; oversized files rejected; filenames are stored safely; files are correctly retrievable and linked.

7.3.5. Reminder generation + opt-out

Methods: stored procedure tests, duplicate tests, eligibility tests.
Expected outcomes: reminders generated only for eligible students; opt-out students excluded; duplicates prevented; reminders scheduled at correct due dates.

7.3.6. Insurance module

Methods: claims submission tests, processing tests, report tests.
Expected outcomes: valid claims are recorded; staff can update status; students see updates; expiry report reflects correct policies.

7.3.7. Exit + clearance module

Methods: full workflow tests and cascading integrity tests.
Expected outcomes: exit requests create exit cases; clearance record creation and unit clearances behave correctly; delete/update operations do not leave orphan records; staff actions are logged and visible to staff.

7.3.8. Notifications and reporting

Methods: correctness tests + performance checks.
Expected outcomes: notifications appear for correct student; unread/read behavior correct; reports return accurate results and do not fail for empty datasets.

7.4. Test data strategy

Testing will use a controlled dataset that includes:

- Students from each subtype (PC/UG/PG)
- Students with one nationality and with multiple nationalities
- Students with active visa, expired visa, and no visa record
- Students opted out of visa reminders and students eligible for reminders
- Visa renewal applications in different states (submitted, pending, approved, rejected)
- Multiple documents per application (valid and invalid upload attempts)
- Insurance policies near expiry and not near expiry
- Exit cases at multiple stages (new request, clearance in progress, completed)

Expected outcome: test scenarios are repeatable and cover both typical and edge conditions without relying on random data.

7.5. Non-functional testing

7.5.1. Security testing

Method: targeted adversarial tests (manual penetration-style checks).
Expected outcomes: unauthorized access blocked; file upload restrictions enforced; sessions cannot be reused after logout; SQL injection attempts fail (prepared statements or strict validation); sensitive pages cannot be opened without session.

7.5.2. Performance testing

Method: basic load simulation (multiple users performing core tasks) and query time evaluation for reports.
Expected outcomes: normal pages respond quickly; reports complete within acceptable time; no database “lock-ups” or long freezes under typical workload.

7.5.3. Reliability and error-handling testing

Method: fault injection (simulate missing DB connection, corrupted input, missing files).
Expected outcomes: user receives safe error messages; no sensitive details leak; system remains consistent and usable afterward.

7.5.4. Usability testing

Method: task-based evaluation during UAT.
Expected outcomes: users can complete key tasks without guidance; messages are clear; forms provide adequate feedback.

7.6. Regression testing approach

After any code or schema change, a regression suite will be re-run covering:

- login + role access
- registration + subtype/nationality integrity
- visa renewal + document upload
- reminder generation + duplicate prevention
- insurance claims + staff processing
- exit workflow + clearance integrity
- reports generation

Expected outcome: new changes do not break previously working functionality.

7.7. Exit criteria

Testing will be considered complete when:

- all high-priority workflows pass end-to-end
- stored procedures produce correct results for normal and edge cases
- no critical security bypass or data integrity issue exists
- UAT participants confirm system meets operational expectations
- regression suite passes after final fixes